PROJECT BRIEF



ZoneGuard

Agentic Visual Safety Inspector

Turning existing workplace video into evidence-backed safety investigations with OpenCV 5 and an adaptive inspection loop.

SEE

Detect people and track zone occupancy

INVESTIGATE

Collect focused visual evidence adaptively

EXPLAIN

Report events with traceable evidence

DEVELOPMENT BRIEF - STATUS AS OF 27 OCTOBER 2026

Local vision workflow partially verified; AWS deployment and live frontend integration remain pending.

Prepared to explain the project concept, architecture, technology choices, validation, and limitations to judges.

01 / THE THESIS

Executive summary

ZoneGuard is a video investigation tool for workplace safety teams. It uses computer vision to identify possible safety-zone events, then collects targeted evidence before presenting a defensible result.

> **One-sentence pitch**
> ZoneGuard turns recorded workplace CCTV into an auditable investigation: detect a possible zone violation, gather relevant visual evidence, verify the transition where possible, and return ALERT, CLEAR, or REVIEW.

The problem

Many workplaces already have cameras, but footage is often reviewed only after a near-miss or incident. Manually scanning a long recording is slow, attention-intensive, and hard to reproduce. Basic motion alarms can also produce noisy alerts because a person merely appearing inside a polygon is not necessarily proof of a new entry or a safety violation.

The proposed answer

- **Perceive:** process frames using OpenCV 5 and a YOLOX ONNX detector; maintain track and zone context.
- **Investigate:** use the current visual evidence to choose the next useful operation, such as extracting relevant frames or verifying a zone transition.
- **Explain:** provide timestamps, zone information, evidence images, an annotated video, and a machine-readable report.
- **Escalate uncertainty:** return REVIEW when evidence is incomplete rather than inventing certainty.

> **Why this is more than object detection**
> The intended innovation is the feedback loop between perception and investigation. A first-pass detection is a hypothesis; later visual checks attempt to confirm, reject, or leave it unresolved. The agentic behavior must be demonstrable through real tool calls and their trace, not just a chat explanation.

Initial target users

Warehouse and logistics safety leads first; manufacturing floor supervisors and industrial safety teams are adjacent use cases. The initial scope is retrospective video review with user-defined polygons, not autonomous control of equipment.

02 / WHY BUILD IT

Problem, users, and intended impact

ZoneGuard is aimed at a narrow, understandable safety workflow where visual evidence and transparent uncertainty matter.

What operators need

Find the relevant moment

Avoid manually scrubbing through an entire recording to locate a possible event.

Know which zone is involved

Let the operator draw named polygons for restricted areas or pedestrian / vehicle interaction zones.

See why the system raised a concern

Present relevant frames, timestamps, track history, and the actual sequence of analysis steps.

Avoid false certainty

Separate a model detection from a verified event, and reserve REVIEW for ambiguity or incomplete analysis.

Keep the output usable

Export a compact incident report and annotated MP4 that can be reviewed by a human.

Example scenarios in scope

Scenario	What ZoneGuard attempts to establish	Expected handling
Pedestrian enters a restricted zone	Whether a track transitions from outside to inside the polygon.	ALERT only if the configured rule is verified; otherwise REVIEW.
Person walks close to a zone boundary	Whether the footpoint actually crosses, rather than jitters along the edge.	Avoid counting boundary noise as a confirmed entry.
Track is temporarily lost	Whether the evidence gap is short enough to preserve continuity.	Gap-aware logic; REVIEW if identity/transition is ambiguous.
Clip starts with a person already inside	Whether the system can infer entry from the clip.	Do not claim a new entry without observed transition evidence.
No qualifying event detected	Whether the required analysis ran successfully and found no qualifying event.	CLEAR only when analysis is complete; REVIEW if it is not.

> **Intended impact, not an unproven claim**
>
> The intended benefit is faster and more consistent video triage with evidence a human can audit. Time saved, lower false-alarm rates, and safety outcomes still need measurement; this brief does not claim those outcomes have already been demonstrated.

03 / PRODUCT WORKFLOW

How a video investigation works

The interface and backend are designed as a single workflow, even though they are being built and deployed separately.

1. Upload video

An operator selects a supported clip. The backend validates it and reads metadata such as dimensions and frame rate.

2. Define zones

The operator draws one or more polygon zones over a representative frame. Coordinates are mapped to the source-video geometry.

3. Detect and track

OpenCV DNN runs the YOLOX ONNX detector. Detections are associated into tracks to support temporal reasoning.

4. Check occupancy and transitions

Footpoint location is used for polygon occupancy. Entry/exit events depend on observed state transitions, not just a single in-zone detection.

5. Investigate adaptively

When an event is uncertain or potentially important, the controller can request relevant frames and invoke a verification tool using prior observations.

6. Decide and report

The system produces ALERT, CLEAR, or REVIEW according to available evidence and the configured event rules.

7. Human review and export

The operator inspects the timeline, evidence, trace, annotated MP4, and JSON report; reviewer actions are only persisted if the matching backend capability exists.

> **Decision semantics**
> ALERT = qualifying event established. CLEAR = required analysis completed and no qualifying event observed. REVIEW = evidence is ambiguous, incomplete, or the required analysis did not finish. These states must come from actual backend outcomes.

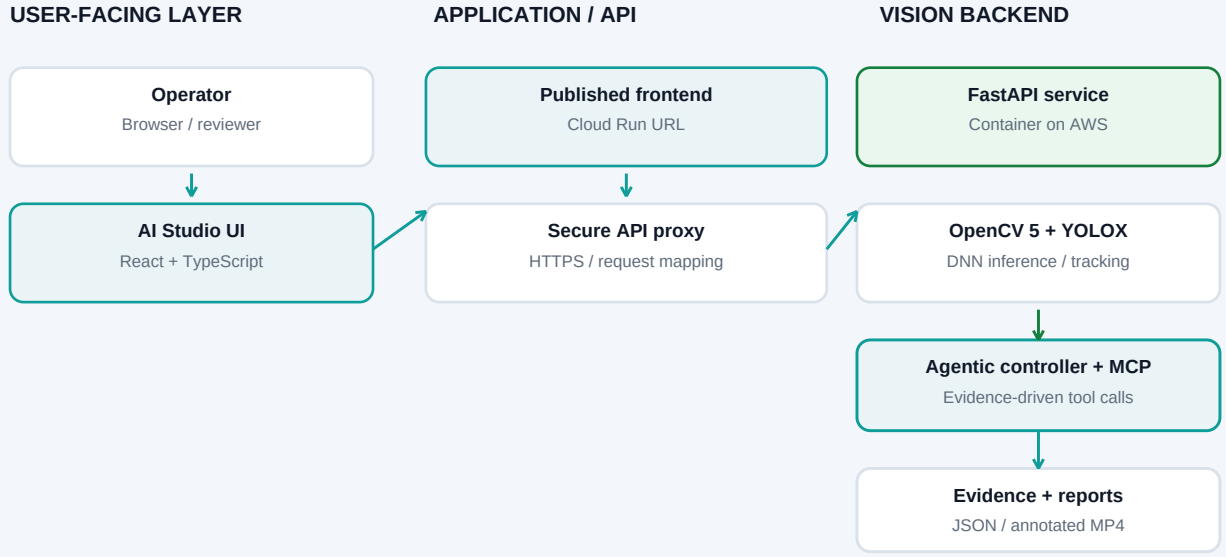
Output artifacts

The project already reports a locally generated annotated video (`data/test-annotated.mp4`) and JSON incident reports as part of the implementation. The exact evidence included in a given report must match the actual analysis run; no placeholder event counts or synthetic tool traces should be presented as real output.

04 / TWO LAYERS, ONE PRODUCT

System architecture

The frontend is being built in Google AI Studio; the vision backend is being developed locally with Codex and is intended to run on AWS. The application becomes end-to-end only after their API contract is connected and tested.



Target architecture. AWS hosting and cross-cloud integration are planned, not yet verified as deployed.

Layer	Responsibility	Current status
Frontend (Google AI Studio)	Upload and preview UI, polygon editor, investigation timeline, evidence viewer, result states, report controls.	A separate frontend project exists; complete live API integration is pending.
API boundary	Secure HTTPS requests, schema mapping, upload handling, result retrieval and downloads.	API client/proxy design is intended; must be matched to the deployed OpenAPI schema.
Vision backend (AWS target)	FastAPI routes, OpenCV 5, YOLOX inference, tracking, zone rules, agent controller, MCP tools, report/video generation.	Local components reported; AWS deployment not yet verified.
Artifacts / observability	Store or serve source videos, evidence, JSON reports, annotated MP4, logs, run identifiers.	Generated local artifacts reported; durable cloud storage and deployed monitoring remain to be implemented/verified.

Critical architecture distinction

Google AI Studio publishing deploys the web app through Google Cloud Run, while AWS is the intended home for the OpenCV/FastAPI backend. These are separate cloud deployments. A frontend URL alone does not demonstrate that the OpenCV component runs on AWS.

05 / TECHNICAL CHOICES

Technology stack and why each part exists

The stack is deliberately split: the browser handles interaction; Python handles video analysis; cloud infrastructure serves the backend and artifacts.

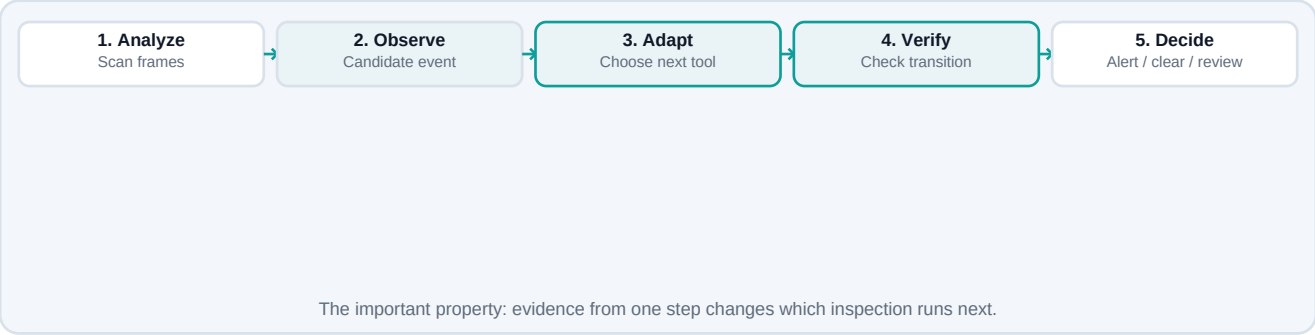
Technology	Role in ZoneGuard	Why this choice	Status
OpenCV 5.0.0	Frame handling, image processing, DNN inference and annotated video generation.	Core contest requirement; a mature computer-vision library with video and inference primitives.	Reported local verification
YOLOX ONNX (OpenCV Zoo)	Detects objects, including people, in video frames.	A pre-trained detector avoids training a custom detector from scratch; ONNX supports portable inference.	Inference reported; task-specific accuracy unmeasured
OpenCV DNN	Loads and executes the ONNX detector.	Keeps the detector in the OpenCV runtime instead of requiring a separate inference service.	Reported local inference
Python	Orchestrates the video pipeline, zone logic, tracking, controller, and reports.	Good ecosystem for computer vision, video processing, testing, and backend integration.	Implemented locally
FastAPI	HTTP API for health, upload and analysis operations.	Typed request/response models and OpenAPI make frontend integration and testing easier.	Local API files reported; cloud not verified
MCP server + client	Exposes named tools and lets the controller invoke investigation actions.	Makes tool boundaries explicit and can support inspectable multi-step investigation.	Seven tools register; Windows stdio initialization still hangs
React + TypeScript	Frontend pages, polygon editor, timeline and result panels.	Component reuse and types help keep UI/API integration maintainable.	AI Studio frontend project; integration pending
HTTP/HTTPS + JSON	Application request/response contract and incident report format.	Simple interoperable boundary between frontend and backend, with structured outputs for debugging.	Target contract to be finalized against real API
Docker	Packages the Python API and its dependencies for deployment.	Reduces differences between local and cloud runtime; supports repeatable deployment.	Dockerfile reported; cloud build not verified
Git + GitHub	Version control and judge-accessible source repository.	Makes code history, setup instructions and reproducible implementation available for review.	Repository created; final code and completeness should be verified
AWS container service	Hosts the substantive vision backend.	Meets the required AWS component while keeping the Python/OpenCV runtime under direct control.	Planned, not deployed
Google AI Studio / Cloud Run	Builds, previews, and publishes the frontend web application.	Accelerates frontend iteration and provides a published URL without mixing UI work into the vision backend.	Frontend workflow; final deployed integration pending
pytest + evaluation scripts	Regression checks and event-level evaluation.	Makes behavior reproducible and surfaces regressions as algorithms change.	Six tests reported passing; labelled video dataset absent

Model provenance note: the YOLOX model referenced by this project is from OpenCV Zoo. The OpenCV Zoo model README describes the YOLOX variant and lists Apache 2.0 licensing for that model directory; the project should retain the exact model provenance and SHA-256 in its own README. See reference [3].

06 / EVIDENCE CHANGES THE NEXT ACTION

The agentic vision idea

ZoneGuard aims to implement active visual investigation rather than a fixed detection pass followed by a language-model summary.



The important property: evidence from one step changes which inspection runs next.

Why an agentic loop matters here

A first detection tells the system that a person may be inside a zone. It does not necessarily establish how the person got there, whether the transition happened during the clip, or whether tracking gaps made the conclusion unreliable. ZoneGuard therefore treats the initial output as a candidate observation and uses it to decide what to inspect next.

Stage	Example tool / operation	Evidence dependency
Observe	`analyze_video`	Produces the initial detections, tracks, occupancy states or candidate events.
Target	`extract_relevant_frames`	The candidate timestamp and uncertainty determine which nearby frames are worth inspecting.
Verify	`verify_zone_crossing`	Uses the selected evidence, track history, footpoint positions and polygon geometry to evaluate the transition.
Conclude	`generate_incident_report`	Uses actual verification output to record the finding, evidence and limitations.

MCP is a tool integration protocol, not the agent itself. The controller is responsible for deciding which tool to call, with which arguments, and when to stop. MCP provides a standard boundary around tool discovery and invocation. The official Python SDK supports stdio, Streamable HTTP and SSE transports; the appropriate transport must be tested in the deployment environment. See reference [4].

> **What has and has not been proven**
>
> The project team reports a real investigation trace through analysis, frame extraction, transition verification and report generation. However, the external MCP stdio subprocess smoke test still hangs during initialization in the Windows development environment. Therefore, internal controller/tool execution has been exercised, but external stdio transport remains unverified.

Design rule: never fabricate tool traces or confidence values. A tool call should include actual inputs and outputs, timestamps, duration, errors, and a reason for the next action. Limit loops, handle failures and make termination explicit.

07 / CLEAR DEPLOYMENT RESPONSIBILITIES

Why AWS for the backend and AI Studio for the frontend

The two platforms are chosen for different responsibilities, not as competing ways to run the same code.

Why AWS for the OpenCV backend

- **Competition fit:** the OpenCV AI Competition 2026 requires OpenCV 5 for substantive visual analysis and a meaningful component running on AWS. Hosting the real inference and investigation API on AWS gives the cloud deployment a substantive role, rather than serving only a static page. [1]
- **Runtime control:** the Python container needs compatible OpenCV, model files, video libraries, memory and CPU settings. Container hosting lets the project package and test these dependencies together.
- **Service boundary:** FastAPI exposes a stable HTTPS API to the UI; the backend can later add storage, job status, logs, and scaling without redesigning the frontend.
- **Operational path:** Amazon ECS Express Mode is one possible target for a containerized API; AWS documents that it can configure a Fargate service, URL, load balancer with TLS, scaling, monitoring and networking. It is a candidate architecture, not a deployed project component yet. [5]

Why Google AI Studio for the frontend

- **Fast UI iteration:** Build mode can generate and refine a React web interface from prompts while providing a live preview.
- **Separate ownership:** the UI can be refined independently from the Python vision pipeline developed in Codex.
- **Publishable URL:** Google documents deployment from AI Studio Build mode to Cloud Run. That gives the frontend a public web endpoint; it does not automatically deploy the separate AWS backend. [2, 6]
- **Integration flexibility:** a typed API client and a server-side proxy can map UI actions to the real FastAPI routes once the API schema is stable.

> **Cross-cloud connection requirements**
>
> The live application must use HTTPS, an explicit CORS policy or secure server-side proxy, bounded upload sizes, request timeouts, and safe artifact URLs. Keep credentials in server-side configuration. Cross-cloud networking, latency and possible data-egress costs should be checked before production use.

Recommended deployment sequence

1. Containerize and test the FastAPI/OpenCV backend. 2. Deploy it to AWS and verify the health route externally. 3. record the actual OpenAPI schema. 4. Configure the frontend proxy/base URL. 5. test upload, analysis, job/result retrieval and report downloads end to end. 6. verify logs, permissions, storage lifecycle and error handling.

08 / WHAT HAS BEEN CHECKED SO FAR

Implementation and verification status

This status snapshot is based on the latest development report provided by the project team on 27 October 2026. It distinguishes test results from unresolved integration work.

Area	Reported result	Interpretation
OpenCV runtime	OpenCV 5.0.0 verified.	Local environment matches the stated OpenCV 5 target.
Model integrity	YOLOX ONNX SHA-256: `DCAAE0AAA2FEA4167F89235EE340EB869D3707B25712218D4C7CE921AC90E2BA`.	Model file integrity can be checked against the reported digest.
Inference	YOLOX ONNX inference through OpenCV DNN completed.	Demonstrates detector execution on the local environment; not evidence of general safety-event accuracy.
Real-video run	Supplied GIF processed; annotated MP4 `data/test-annotated.mp4` generated.	A basic end-to-end local media path has been exercised.
Agent investigation	Trace: `analyze_video` -> `extract_relevant_frames` -> `verify_zone_crossing` -> `generate_incident_report` -> complete.	A reported multi-step trace shows evidence-dependent stages being executed.
Final decision	`REVIEW` for the supplied GIF.	Appropriate because an outside-to-inside transition was not established from the available evidence.
Automated tests	`6 passed in 0.64s`.	Current regression suite passes; its coverage and size do not prove production accuracy.
Evaluation dataset	`status: insufficient_data`, `videos: 0`.	No labelled evaluation clips are yet included; precision, recall and F1 are not available.
MCP stdio transport	Server registers seven tools; subprocess smoke test hangs during initialization on Windows.	External MCP stdio transport remains unverified.
AWS deployment	No verified AWS deployment or credentials reported.	Target architecture only; the cloud service and public endpoint still need deployment and test.
Frontend integration	AI Studio frontend is a separate project; complete API integration remains pending.	A published frontend, if available, must not be presented as the complete system until the backend is connected.

No fabricated performance metrics

ZoneGuard has not yet reported project-specific precision, recall, F1, false-alert rate, or measured time saved because the annotated evaluation dataset is not available. Published benchmark figures for the upstream detector are not ZoneGuard event-detection metrics.

09 / MEASURE BEFORE MAKING CLAIMS

How the project will be evaluated

A useful safety system must be assessed on events, not only on whether a neural network can return bounding boxes.

Dataset plan

Build a small, manually labelled collection of permitted workplace-style clips, with event timestamps and zone IDs. Include positive entries, non-events, boundary jitter, occlusion, long tracking gaps, initial occupancy and multiple people / zones. Document video provenance and exclude private footage unless permission and appropriate handling are in place.

Evaluation measures

Measure	Meaning	Why it matters
Event precision	TP / (TP + FP)	Of the events ZoneGuard reports, how many match a labelled event?
Event recall	TP / (TP + FN)	Of the labelled events, how many did ZoneGuard identify?
F1 score	Harmonic mean of precision and recall.	A combined summary when both precision and recall are defined.
False alerts per video-minute	False positive event count divided by evaluated video minutes.	Makes alert burden easier to compare across clips of different duration.
Event matching tolerance	A documented time window used to match a prediction to a labelled event.	Prevents ambiguous matching rules from inflating or reducing scores.
Latency / real-time factor	Wall-clock processing duration; processing time divided by video duration.	Shows throughput constraints and helps choose deployment resources.

Evaluation controls

- Define annotation rules before running the model; keep labels separate from prediction files.
- Count event-level true positives, false positives and false negatives using one explicit matching rule and timestamp tolerance.
- Report the number of videos and total video minutes, not just a percentage.
- Keep failed or unreadable clips visible in the report rather than silently excluding them.
- Show representative successes and failure cases, including examples where the right decision is REVIEW.

> **What success would look like**
>
> Not a predetermined accuracy number. Success is a repeatable evaluation workflow, a clearly documented dataset, measured error modes, and honest evidence about where the system works and where human review remains necessary.

10 / SAFETY IS CONSEQUENTIAL

Limitations, privacy, and responsible use

ZoneGuard is a decision-support tool. It should help people find and inspect relevant footage, not replace safety staff or automatically discipline workers.

Limitation / risk	Mitigation and honest claim
Occlusion, low resolution, camera angle and lighting	Surface evidence quality; measure the effect on missed detections and false alerts. Use REVIEW when the event cannot be established.
Detector confidence versus event confidence	Show these as distinct concepts. A high-confidence person detection does not prove a zone transition occurred.
No labelled performance set yet	Do not publish precision, recall or F1; create a documented evaluation set first.
No real-world distance calibration	Pixel distance is not a measurement in metres without camera geometry and calibration. The initial scope does not claim physical-distance estimates.
No PPE model in the current scope	YOLOX person/object detection alone does not establish hard-hat, vest, or other PPE compliance.
Privacy and worker surveillance	Use authorized footage; minimize retention; restrict access; avoid face recognition or identity claims; define deletion policies before production.
Human consequences of false alerts	Keep a reviewer in the loop; record reviewer decisions separately; avoid automatic disciplinary or employment decisions.
Security and file handling	Limit upload type/size, validate requests, avoid arbitrary file paths, protect private URLs, and prevent outputs from different runs from overwriting one another.
Cross-cloud deployment	Use HTTPS, a minimal API surface, server-side secrets, explicit CORS / proxy configuration, operational logs and cost monitoring.

Responsible-use commitment

ZoneGuard should be used to prioritize human review and create a traceable record of what the software observed. It should not be described as a certified safety system, guaranteed detector, or substitute for site-specific risk assessment.

11 / NEXT MILESTONES

Roadmap and judge walkthrough

The next work is reliability and integration, not broadening the feature list.

Milestone	Work	Exit condition
A. Resolve MCP transport	Diagnose the Windows stdio initialization hang; verify initialize, tool discovery, invocation, error handling and shutdown with timeouts.	A repeatable external MCP subprocess smoke test passes, or the deployment-compatible transport is explicitly selected and tested.
B. Label evaluation clips	Create a small permitted labelled dataset and run the event-level evaluator.	Machine-readable output plus a report that includes dataset size, metrics where valid, failures and latency.
C. Finish frontend contract	Inspect the actual FastAPI OpenAPI schema; map UI requests and response models to it.	A frontend user can configure zones and submit a real investigation, not a mock request.
D. Deploy backend to AWS	Package service and model; deploy; verify public health and analysis routes; configure storage/logging appropriately.	An externally reachable HTTPS backend runs the actual OpenCV processing component.
E. End-to-end verification	Connect frontend and AWS; test upload, zones, analysis, event review, evidence, video and JSON downloads.	A complete user journey works from the published web URL against the actual backend.
F. Judge package	Architecture diagram, dependencies, setup guide, short demo video, evaluation evidence and limitations.	A judge can run or inspect the system, understand the design, and reproduce the reported checks.

Suggested five-minute demonstration

Time	Show	Narration goal
0:00-0:30	Problem and product	Existing CCTV is valuable but expensive to review manually; ZoneGuard finds candidate moments and organizes evidence.
0:30-1:15	Upload and draw zones	Show a real clip, multiple polygons and the actual submitted configuration.
1:15-2:30	Run investigation	Show real OpenCV processing and the evidence-driven tool sequence.
2:30-3:30	Inspect evidence and decision	Play annotated video, open timeline evidence, explain why the result is ALERT / CLEAR / REVIEW.
3:30-4:15	Architecture and deployment	Distinguish the AI Studio frontend from the AWS OpenCV backend; show the real endpoint / logs.
4:15-5:00	Evaluation and limitations	State measured results only; show at least one failure or REVIEW case and explain next steps.

The judge demo should use actual outputs from the same recorded run shown in the UI. If a component is not deployed or tested, say so directly rather than switching to a static mockup.

12 / JUDGE REFERENCE

Summary and references

ZoneGuard is designed around a specific thesis: safety video review improves when visual evidence is used to guide the next inspection and every conclusion remains traceable to the evidence available.

Core contribution

The product combines OpenCV-based video perception, polygon-zone event logic, an evidence-driven multi-step investigation controller, and a review-oriented web interface. The differentiator is not the detector alone; it is the inspectable path from candidate observation to verified or unresolved event.

Primary references

[1] OpenCV AI Competition 2026, powered by AWS. Requirements, suggested areas, submission requirements and judging criteria. https://opencv26.devpost.com/

[2] Google AI for Developers. Build apps in Google AI Studio: Build mode, React/web app workflow, preview and deployment options. https://ai.google.dev/gemini-api/docs/aistudio-build-mode

[3] OpenCV Zoo. YOLOX model documentation, model variant details, demo instructions and model-directory license. https://github.com/opencv/opencv_zoo/blob/main/models/object_detection_yolox/README.md

[4] Model Context Protocol. Official Python SDK documentation: MCP servers/clients and supported transports. https://py.sdk.modelcontextprotocol.io/

[5] Amazon Web Services. Amazon ECS Express Mode overview: container-based service deployment, HTTPS, scaling, monitoring and networking. https://docs.aws.amazon.com/AmazonECS/latest/developerguide/express-service-overview.html

[6] Google AI for Developers. Deploying from Google AI Studio to Cloud Run. https://ai.google.dev/gemini-api/docs/aistudio-deploying

[7] OpenCV. Official releases page, listing OpenCV 5.0.0. https://opencv.org/releases/

Project status note

This brief is based on the implementation reports available on 27 October 2026. It is a technical explanation of the project direction and current reported state, not an independent audit of the source repository. Reported local tests and files should be re-run from the submitted code; AWS deployment, live frontend integration, and MCP stdio transport must be verified before any claim that the complete cloud application is operational.